

Bug Fix Report - Sample Deliverable

Sample deliverable - "debug your Python bug" gig

A real bug, anonymized into a client-facing write-up. This is the format and depth every "debug" order ships with.

Bug fix report - "my automation silently skips every item"

Symptom (what the client reported)

- "My review bot stopped processing anything. It runs without errors, the logs look normal, but it silently skips every item as 'wrong project'."

No crash. No traceback. Just a system that looks alive but does nothing - the worst kind of bug, because it fails without telling you.

Root cause (what was actually wrong)

The script read rows from a SQLite table by positional column index:

```
cols = [r[1] for r in conn.execute("PRAGMA table_info(sessions)")]
# ...
row = conn.execute("SELECT source, title, ended_at, last_activity_at, cwd "
                  "FROM sessions WHERE id = ?", (id,)).fetchone()
# ...later, walking the row tuple by position:
cwd = row[4]    # <-- assumed column #4 was cwd
```

Someone added a `last\_activity\_at` column in the middle of the schema. The `SELECT` list was updated, but the code that read the result still walked the tuple by hard-coded position. So column index 4 - now `last\_activity\_at` (a timestamp) - was being read as the project directory.

Every session therefore looked like it belonged to a different project than it actually did, so the "is this mine?" check silently rejected **all** of them.

The tell: the bug only appeared after a schema change, and it never threw - it just mis-read data.

The fix

Stop addressing columns by position. Read them by name instead, so a column added or reordered can never shift an index:

```
cols = session_columns(state_db)          # {name, ...} from PRAGMA
select = ["source", "title"]
for c in ("ended_at", "last_activity_at", "cwd"):
    if c in cols:
        select.append(c)
row = conn.execute(f"SELECT {' '.join(select)} FROM sessions "
                  f"WHERE id = ?", (id,)).fetchone()
rec = dict(zip(select, row))               # named, order-independent
cwd = rec.get("cwd") or ""
```

Two properties this guarantees:

1. Order-independence - the row is keyed by column name, not index.
2. Schema-graceful - a column that's missing is simply absent from the dict (`rec.get("cwd")` returns `None`), never silently shifted.`

Regression test

A test now builds the table with columns in a non-canonical order and asserts the reads stay correct - so if anyone reorders the schema again, the test fails loudly instead of the bot silently skipping everything:

```
# columns deliberately shuffled; reads must still return cwd correctly
assert load_session(db, sid)["cwd"] == "/expected/path"
```

What you got

- The root cause, not a patch (a patch to index 4 would have broken again on the next schema change).
- A fix that removes the entire class of bug (positional reads), not just this instance.
- A regression test so it can't come back without a loud failure.

This is a representative sample: the actual bug, code, and fix are real, edited only to remove project-specific context.